

Test Case ID	How I fixed the issue	Evidence (Key Words)
Example	"At first, when I clicked the Add button, nothing happened. I realised I forgot to connect the command= in the Button to the function. I fixed this by adding command=add_task."	<code>command = function()</code>
2	At first when I pressed the the left and right corresponding keys (A and D), the player did not accelerate nor decelerate its current velocity and continued to move. I realised that I forgot to add a condition that ensures that the velocity only equals zero when the player is near an appropriate value. I fixed this by adding a condition to round the float of velocity to zero when necessary.	<code>if velocity.z > 0: velocity.z = 0</code>
4	At first when I tested the rotation of the player character to face the mouse cursor it did not change it rotation appropriately. I noticed that dependent on the viewport and the 3D world were two completely different vector types due to the origin being the centre for the 3D and the origin being the topleft of the viewport for 2D. I fixed this by applying a raycast from the camera to get a relative position for the player to reference to instead of using the mouses' relative position on the viewport 2D window.	<code>mouse_loc.text = "Mouse Location: " + str(get_viewport().get_mouse_position()) player_direction.text = "Player Location: " + str(self.position) var space_state = get_world_3d().direct_space_state var mousepos = get_viewport().get_mouse_position() var origin = camera_3d.project_ray_origin(mousepos) var end = origin + camera_3d.project_ray_normal(mousepos) * 1000 var query = PhysicsRayQueryParameters3D.create(origin, end, 2) query.collide_with_areas = true var result = space_state.intersect_ray(query) result.position.y = position.y look_at(result.position)</code>
5	At first when I fired a projectile at the enemy character body, I noticed that it was not changing colour to indicate that it was recieving a hit input. I noticed that I did not give enough time for the appearance to change then revert back to its original state turning to its default in a short tick rather than a few seconds. I fixed this by adding an await parameter that instantiates a temporary timer than deletes it resulting in a cleaner result.	<code>func _on_dummy_hitbox_area_entered(area: Area3D) -> void: print("registered") if area.name == "Bullet_collider": mat.albedo_color = Color(0.871, 0.157, 0.451, 1.0) READY = false await get_tree().create_timer(1.5).timeout mat.albedo_color = Color(0.255, 0.412, 0.71, 1.0)</code>
7	At first when I hit left click, the player's bullets were not firing in a straight trajectory. I noticed that the bullets were colliding with the reference marker and the player model from the beginning. I fixed this by temporarily changing the player mask to infer that the bullet cannot hit its own initiator of bullet fire.	<code>(nil)</code>
8	At first when firing the bullet I noticed that the bullet did not fall downwards at a constant rate. This was because I did not apply a relevant gravity constant for the bullet to refer to changing its velocity in that direction. I fixed this as according.	<code>func _process(delta: float) -> void: global_position += -global_transform.basis.z * speed * delta global_position += -global_transform.basis.y * gravity * delta</code>

14 At first when counting down from the timer, the game did not proceed with the countdown upon the label interface. I had noticed that although I am using a process function, it will be called every game tick. This means that the function was being called constantly therefore locking the timer. I fixed this by adding a condition to check if the function has been called and will proceed when necessary

```
if minutes == 0 and seconds == 0 and ready_next_phase == false:
    if game_state == "post_game":
        counting = false
        ready_next_phase = true
        modifiers_proceed()
    elif game_state == "pre_game":
        minutes = in_game_minutes
        seconds = in_game_seconds
```

19 At first when loading the game as host, the camera caused the game to crash. I noticed that the directory that the player needed to reference for the aiming requirements was shifted after I changed the directories of the players and the level scene. In order to fix this I changed the camera localisation to be global so all player character body nodes could access its properties.

(nil)

20 At first when testing the capabilities of the local hosting x client service, I noticed that once the players joined under client, they did not properly change their respective modulattive colour respective to joining server priority. This was mainly because I did not provide a proper reference for the player's player body to change colour. Therefore I fixed this by providing a correct reference to the colour change as well as calling an @rpc annotation to allow other players to see this change

```
@rpc("call_local")
func colour_change():
    if name == GameManager.players[0]:
        mat.albedo_color = Color(0.0, 1.0, 1.0, 1.0)
    elif name == GameManager.players[1]:
        mat.albedo_color = Color(0.0, 1.0, 0.0, 1.0)
    elif name == GameManager.players[2]:
        mat.albedo_color = Color(1.0, 0.5, 0.0, 1.0)
    elif name == GameManager.players[3]:
        mat.albedo_color = Color(1.0, 0.0, 0.0, 1.0)
```

21 At first when proceeding to the modification selection stage, client players could not see the instantiated cards like the host could. I noticed that the RPC command was not properly locally sharing these cards among the other clients. Therefore I fixed this problem by adding a proper rpc command for all players to see their card options when they reach to the modifier selection stage.

```
@rpc("any_peer", "reliable")
```

22 At first when players reached the modification stage, they did not end up with the same card selection that the other players had. I noticed that in order to fix this problem, I would have to ensure that instead of each client separately calling and deducting their own randomised selection that I would have the server come up with a random string of digits to identify the card ids then be processed upon the clients. I did this as following.

```
@rpc("authority", "call_local", "reliable")
func card_loader(card_ids: Array[int]) -> void:
    var index := 0

    for row in range(2):
        var row_load := HBoxContainer.new()
        $VBoxContainer.add_child(row_load)
        row_load.add_theme_constant_override("separation", 70)

        for column in range(4):
            var card_id: int = card_ids[index]
            var card_data: Dictionary = CardData[card_id]
            index += 1

            var loaded_card = Card_template.instantiate()

            loaded_card.setup_card(card_id, card_data)
            loaded_card.card_selected.connect(_on_card_selected)

            row_load.add_child(loaded_card)
```

26	At first when testing the capabilities of the mouse cursor, I noticed that the custom png was not changing its relative position to the viewport location of the mouse. I noticed that the reference of the mouse cursor had changed when it entered a 3D viewport. In order to fix this I changed the global position of the mouse cursor to be local to the 2D viewport to easily determine the relative locations between the two.	<pre>func _process(delta: float) -> void: if !is_multiplayer_authority(): return global_position = get_global_mouse_position()</pre>
27	At first when testing the host to change colour upon joining the game, I noticed that the colour of the cursor was not changing mainly due to an improper reference of modulate vs self_modulate. I fixed this by changing as such.	<pre>func colour_change(player_index: int): #print("My name: ", name) #print("Index received: ", player_index) match player_index: 0: modulate = Color.CYAN 1: modulate = Color.GREEN 2: modulate = Color.ORANGE 3: modulate = Color.RED func colour_change(player_index: int): #print("My name: ", name) #print("Index received: ", player_index) match player_index: 0: self_modulate = Color.CYAN 1: self_modulate = Color.GREEN 2: self_modulate = Color.ORANGE 3: self_modulate = Color.RED</pre>
28	At first when testing for multiplayer compatibility, I noticed that the colour of the cursors local to the other clients were not changing respective to the player that had a cursor on their screen. I noticed that the rpc annotation was not properly signalled amongst player clients, since it was only called when the first players joined the game causing newer players to have a known reference of older player clients. Therefore I fixed this by ensuring that the player mouse decal would always check for changes in player counts then send a signal to announce that a new player has joined and that theyre already players in the lobby.	<pre>@rpc("call_local", "any_peer") func colour_change(player_index: int): #print("My name: ", name) #print("Index received: ", player_index) match player_index: 0: self_modulate = Color.CYAN 1: self_modulate = Color.GREEN 2: self_modulate = Color.ORANGE 3: self_modulate = Color.RED</pre>
33	At first when testing the card modifiers, I noticed that as the player clicked the card, they would not be able to receive the benefits of the card. In order to fix this I added a multiplayer.get_unique_id() inbuilt function to reference which player was pressing the card. Therefore I fixed this issue with proper referencing upon the player's stats.	<pre>func _on_gui_input(event: InputEvent) -> void: if event.is_action_pressed("left_click"): if enabled and !GameManager.player_chosen: vanity.rpc() modifier_apply() elif !enabled: print("disabled")</pre>

36

At first when testing the fire rate increase of the gun that players had it did not increase in fire rate. I noticed that when creating the reference constant for the variable change in fire rate, I did not correctly state a proper increase in fire rate values. Therefore I fixed this issue by changing the value references and values to make the changes seem more significant and less negligible.

```
await get_tree().create_timer(  
    1.9 + (0.9 * attack_speed_modifier)  
).timeout  
  
await get_tree().create_timer(  
    1.9 - (0.9 * attack_speed_modifier)  
).timeout
```

40

At first when restarting some more tests on the synchronisation of player clients, they occasionally did not sync up properly. I noticed that once the server had finished the arena game stage, it would only announce that the card modification stage was available one time no matter if clients were ready or not. I fixed this by adding a condition only then the instantiation of cards would occur when all players were ready.

```
if players_loaded.size() < GameManager.players.size():  
    return
```

41

At first when trying to relay the points to the server from the clients to display as the score for other plays to see on the scoreboard, it failed to execute the correct directory from the player receiving points in the directory. I noticed that the script was unable to correctly reference which player scored a point and which label corresponded to which player. In order to fix this I changed the clients changing the score to the server changing the score. This ensures that the function is not called each time per client making it easier to track the amount of points

```
!multiplayer.is_server():  
  
multiplayer.is_server():
```